

Intelligent Systems

Assignment 1 – Task 1

Maze Problem

Jack Millington

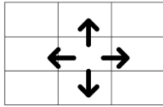
s5405915

1.0 Introduction

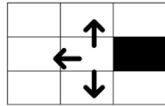
The following maze search problem was solved using an agent that was developed using python, and multiple different search strategies. The goal of this task is to find an optimal path from a starting cell to a destination cell using different search strategies.

There are some rules involved:

1. The agent is allowed to move one cell at a time to an adjacent cell (where no diagonal moves are allowed)



2. If a cell is a wall, then the agent cannot move to that cell. An example can be shown in the image below where the black square represents a wall, so the agent can only move to the other 3 cells.



3. All moves cost the same price

The code reads from a txt file displaying as a maze shown here:

```
1  #####B#
2  ##### #
3  ##### #
4  ##### ##
5  |      ##
6  A#####
```

1.1 Formal Problem Formulation:

State space: Each state represents the agent's current position in the maze, defined as a coordinate pair (row, column) where the agent is located.

Initial State: The starting position marked as 'A' in the maze file, represented as coordinates (start_row, start_col).

Goal State: (goal_row, goal_col).

Actions: From any given state, the agent can perform one of four possible actions:

Move Up: (row-1, col), Move Down: (row+1, col), Move Left: (row, col-1), Move Right: (row, col+1)

Transition Model:

An action is valid if and only if the resulting coordinates are within the maze boundaries ($0 \leq \text{row} \leq \text{height}, 0 \leq \text{col} \leq \text{width}$), AND, the destination cell is not a wall ($\text{maze}[\text{row}][\text{col}] \neq \text{wall}$)

The transition model returns the new coordinates if the action is valid, otherwise the goal is not applicable.

Goal test: A state satisfies the goal test if its coordinates match the goal state coordinates:
 $\text{current_state} == \text{goal_state}$

Path Cost: Each valid move has a uniform cost of 1. The total path cost is the number of steps taken from the initial state to the goal state.

A `reconstruct_path()` helper function eliminates code duplication by tracing back from the goal state to produce the solution path.

```

1      ##      #
2      ##      ##      #
3      #B      #      #
4      #      ##      ##
5      |              ##
6      A#####

Maze:

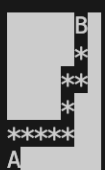


Using algorithm: A*
Solving...
States Explored: 5
U-R-U-U
Total Path Cost: 4
Total Path Length: 4
Solution:



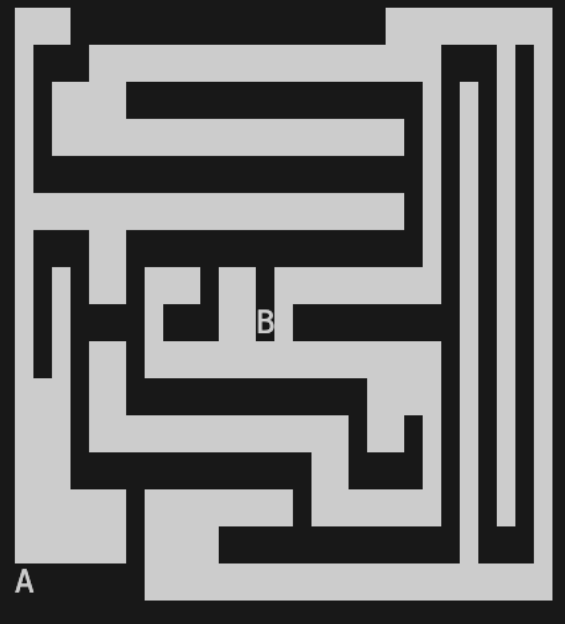
```

3.0 Experiments and Results:

Maze 1:	DFS:	IDS:	A*:
Maze: 	Using algorithm: DFS Solving... States Explored: 11 U-R-R-R-R-U-U-R-U-U Total Path Cost: 10 Total Path Length: 10 Solution: 	Maze:  A Using algorithm: IDS Solving... States Explored: 66 U-R-R-R-R-U-U-R-U-U Total Path Cost: 10 Total Path Length: 10 Solution: 	Using algorithm: A* Solving... States Explored: 11 U-R-R-R-R-U-U-R-U-U Total Path Cost: 10 Total Path Length: 10 Solution: 

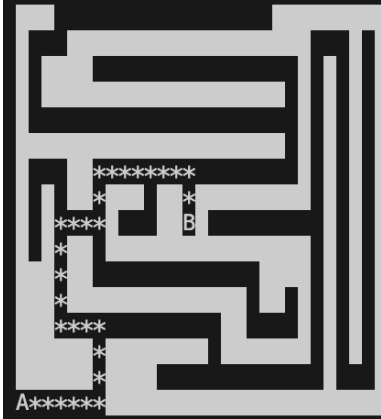
Maze 2:

Maze:



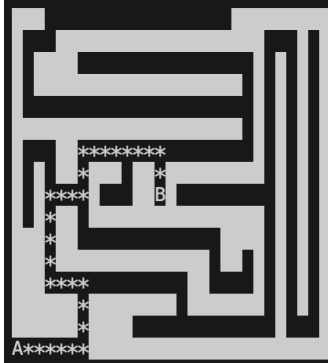
DFS:

```
Using algorithm: DFS
Solving...
States Explored: 194
R-R-R-R-R-R-U-U-U-L-L-L-U-U-U-R-R-R-U-U-R-R-R-R-R-R
Total Path Cost: 30
Total Path Length: 30
Solution:
```



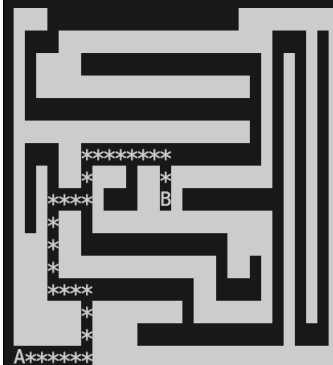
IDS:

```
Using algorithm: IDS
Solving...
States Explored: 910
R-R-R-R-R-R-U-U-U-L-L-L-U-U-U-R-R-R-U-U-R-R-R-R-R-D-D
Total Path Cost: 30
Total Path Length: 30
Solution:
```



A*:

```
Using algorithm: A*
Solving...
States Explored: 59
R-R-R-R-R-R-U-U-U-L-L-L-U-U-U-R-R-R-U-U-R-R-R-R-R-D-D
Total Path Cost: 30
Total Path Length: 30
Solution:
```



Maze 3:	DFS: <pre> Using algorithm: DFS Solving... States Explored: 17 U-R-R-R-R-U-U-R-U-U-L-L-L-D-D-L Total Path Cost: 16 Total Path Length: 16 Solution: </pre>
IDS: <pre> Using algorithm: IDS Solving... States Explored: 18 U-R-U-U Total Path Cost: 4 Total Path Length: 4 Solution: </pre>	A*: <pre> Using algorithm: A* Solving... States Explored: 5 U-R-U-U Total Path Cost: 4 Total Path Length: 4 Solution: </pre>

Maze 4:



DFS:

```
Using algorithm: DFS  
Solving...  
States Explored: 14  
no solution  
Solution:
```



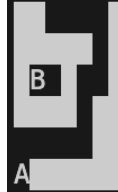
IDS

```
Using algorithm: IDS  
Solving...  
States Explored: 511  
no solution  
Solution:
```

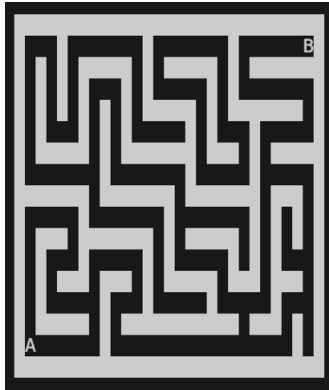


A*

```
Using algorithm: A*  
Solving...  
States Explored: 14  
no solution  
Solution:
```



Maze 5:

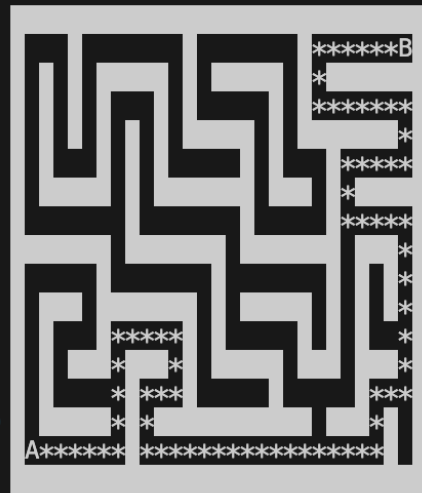


DFS:



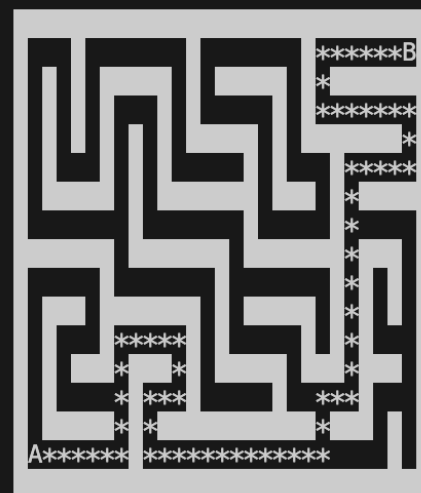
IDS:

```
Using algorithm: IDS
Solving...
States Explored: 5008
R-R-R-R-R-U-U-U-U-R-R-R-R-D-D-L-L-D-D-
R-R-R-R-R-R-R-R-R-R-R-R-R-U-U-R-R-
U-U-U-U-U-L-L-L-L-U-U-R-R-R-U-U-L-L-
L-L-L-L-U-U-R-R-R-R-R
Total Path Cost: 72
Total Path Length: 72
Solution:
```



A*:

```
Using algorithm: A*
Solving...
States Explored: 124
R-R-R-R-R-U-U-U-U-R-R-R-R-D-D-L-L-D-D-
R-R-R-R-R-R-R-R-R-R-R-R-R-U-U-R-R-
U-U-U-U-U-L-L-L-L-U-U-R-R-R-U-U-L-L-
L-L-L-L-U-U-R-R-R-R-R
Total Path Cost: 64
Total Path Length: 64
Solution:
```



4.0 Analysis:

There are clear differences between the 3 different search algorithms. In maze 1, there is only 1 possible path, 11 states were visited when using A* and DFS, however, IDS searched through 61 states for this and therefore is the least efficient for this maze.

In maze 2, all three algorithms found the optimal path, however, DFS, IDS, and A* explored 194, 910 and 59 states respectively. This shows that A* was the most efficient algorithm here, followed by DFS and the IDS.

In maze 3, only IDS and A* found the most optimal path, however the path found by DFS was much longer. A* explored only 5 states compared to 18 when using IDS, therefore, A* is the most efficient algorithm for this maze followed by IDS, and then DFS as it did not find an optimal path.

In maze 4, all search algorithms correctly determined that no solution can be found, however, IDS explored a significantly higher number of states, proving it is prone to inefficient searching.

In maze 5, only A* found the optimal path, while the other two could not. IDS explored a significantly higher number of states compared to the other two algorithms.

Each algorithm will be ranked based off its performance off each maze to determine the best algorithm.

Map	Maze 1	Maze 2	Maze 3	Maze 4	Maze 5
Ranking	DFS: 1 IDS: 3 A*: 1	DFS: 2 IDS: 3 A*: 1	DFS:3 IDS:2 A*:1	DFS:1 IDS:3 A*:1	DFS:2 IDS:3 A*:1

Total: DFS: 9, IDS: 14, A*: 5 (Lower is better).

As A* had the highest ranking, it is clearly the best algorithm out of the 3. This has been tested on a variety of different mazes, including ones with no solution at all.

5.0 Conclusion:

Overall, by gathering the results from multiple different mazes and comparing between the 3 different search algorithms of depth first search, iterative deepening search and A*, it has become clear that A* is the best search algorithm out of the three. This algorithm excelled in both efficiency, and accuracy in finding the optimal solution, throughout varied mazes. Depth first search was found to be overall better than IDS, due to the inefficiency of IDS, even though IDS was more accurate in finding the optimal solution.